A PROJECT REPORT

ON

Dual-Protocol Chat Application Using TCP and UDP

A Course Project report Submitted in partial fulfillment of the Academic requirements for the award of the degree of

BACHELOR OF TECHNOLOGY IN
ARTIFICIAL INTELLIGNECE AND MACHINE LEARNING

Submitted by

S.Sirisha	**23D41A66G9**
T.Harini	**23D41A66H0**
T.Rishwanth	**23D41A66H1**
U.Navya	**23D41A66H2**
U.Akash	**23D41A66H3**

Under the esteemed guidance of

Dr.Sandhya Bolla

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

SRI INDU COLLEGE OF ENGINEERING & TECHNOLOGY(Autonomous)

(Approved by AICTE, New Delhi & Permanently Affiliated to JNTU Hyderabad,Sheriguda (V), Ibrahimpatnam (M),R.R.Dist-501 510)

2025-2026

SRI INDU COLLEGE OF ENGINEERING & TECHNOLOGY

(An Autonomous Institute)

(Approved by AICTE, New Delhi & Permanently Affiliated to JNTU Hyderabad)

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

CERTIFICATE

This is to certify that the Project entitled **"Dual-Protocol Chat Application Using TCP and UDP "** is submitted in partial fulfilment for the award of **Degree of Bachelors of Technology in ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING** of Sri Indu College of Engineering And Technology,Hyderabad for the academic year 2025-2026.This is a record of the project work done by **S.Shirisha(23D41A66G9),T.Harini(23D41A66H0),T.Rishwanth(23D41A66H1),U.Navya(23D41A66H2), U.Akash(23D41A66H3)** which has been approved as it satisfies academic requirements in respect of the work prescribed for **III YEAR,I-SEMESTER of B.TECH course.**

Dr.SANDHYA BOLLA
(Internal guide)

G.UMA MAHESWARI
(Head of the department)

TABLE OF CONTENTS

SNO	TOPIC	PAGE NO
1	ABSTRACT	1
2	INTRODUCTION	2
3	EXISTING SYSTEM	3
4	PROPOSED SYSTEM	4
5	HARDWARE AND SOFTWARE REQUIREMENTS	5
6	IMPLEMENTATION	6
7	RESULT	14
8	CONCLUSION	15

ABSTRACT

This project presents a GUI-based client–server chat application that supports communication using both TCP and UDP transport protocols. The system is designed to demonstrate fundamental concepts of computer networks, socket programming, and protocol behavior through an interactive and user-friendly interface. The server and client applications are implemented using Python and Tkinter, providing a simple and intuitive graphical environment for message exchange.

The application allows users to choose between TCP (connection-oriented, reliable) and UDP (connectionless, faster) before initiating communication. After selecting the protocol, the server and client establish communication over localhost, enabling bi-directional text messaging. Multi-threading is used to ensure that the GUI remains responsive while messages are being sent and received in real time.

This project highlights the practical differences between TCP and UDP, their use cases, and their communication behavior. It is especially suitable for academic learning, computer networks laboratory demonstrations, and understanding protocol-level communication. The system can be further extended for network-wide communication over LAN.

INTRODUCTION

Communication over computer networks plays a vital role in modern distributed systems, enabling data exchange between devices across local and global environments. Among the core mechanisms used for such communication are network protocols, with the User Datagram Protocol (UDP) being one of the most widely used due to its simplicity, speed, and low overhead. UDP provides connectionless communication, making it suitable for real-time transmission where speed is prioritized over guaranteed delivery.

This project focuses on developing a **UDP-based Chat Application** implemented in Python, enhanced with a user-friendly **Graphical User Interface (GUI)** using Tkinter. The system consists of two separate applications—one acting as the **client** and the other as the **server**—both designed to allow users to send and receive text messages seamlessly across the network. The GUI layout includes clearly defined message areas and input sections, ensuring ease of interaction for users.

While traditional command-line chat programs offer functionality, they often lack visual appeal and usability. By integrating a GUI, this project provides an intuitive interface where users can view incoming messages, type responses, and communicate efficiently without needing technical command-line knowledge. The implementation also demonstrates socket programming fundamentals, event-driven message handling, and multi-threading to maintain responsiveness during communication.

This project serves as a practical demonstration of network communication using UDP, graphical interface design, and Python programming, making it highly suitable for academic learning, laboratory experiments, and foundational exploration of networking concepts.

EXISTING SYSTEM

In the traditional networking lab setup, communication between two computers is usually demonstrated using **simple TCP or UDP socket programs** that run only in the **terminal**. These existing systems allow basic message exchange, but they are very limited in terms of usability and user experience.

In the current system:

- Both **client and server run only in command-line windows**.
- Users have to type messages directly in the terminal without any visual interface.
- There is **no proper message display area**—sent and received messages appear mixed together.
- Users often struggle to understand:
    - Where to type the message
    - Which window is client and which is server
    - Whether the message was actually sent or not
- The system does not support labels, formatting, or any graphical elements.
- There is no option to select communication protocol (TCP or UDP) within the interface.
- Overall, the communication feels technical rather than interactive or user-friendly.

Because of these limitations, the existing system is good only for demonstrating basic socket programming but **does not provide a real chat-like experience**. It lacks clarity, ease of use, and visual structure, which makes it difficult for new learners to understand networking concepts smoothly.

PROPOSED SYSTEM

The proposed system introduces an improved communication model that allows users to chat over TCP or UDP using an intuitive **Graphical User Interface (GUI)**. Instead of relying on traditional terminal-based message exchange, the new system provides a modern, user-friendly interface that displays messages clearly and allows users to select the desired protocol before starting the chat.

The objective of the proposed system is to make network communication easier to understand and more interactive. By integrating GUI with socket programming, the system eliminates the confusion of typing commands in different terminals and offers a structured chat environment similar to real-world messaging applications.

What This System Offers

- **User-Friendly Interface:**
  A simple GUI where users can easily see chat messages and know exactly where to type.
- **Protocol Choice:**
  Before starting, the user can choose whether the chat should run on **TCP** or **UDP**.
- **Dedicated Chat Window:**
  Messages are displayed clearly in a scrollable chat area, making the conversation easy to follow.
- **Labeled Input Box:**
  The text-entry field is clearly marked as “Type your message here”, removing any confusion.
- **Send Button:**
  Instead of pressing Enter in a terminal, users can send messages with a simple button click.
- **Real-Time Communication:**
  The chat updates instantly as messages are exchanged through sockets.

HARDWARE AND SOFTWARE REQUIREMENTS

HARDWARE REQUIREMENTS:

Processor :- Intel Core i3 (1st Generation or above)

Hard Disk :- 256 GB

RAM :- 4GB

SOFTWARE REQUIREMENTS:

Operating System :- Windows /MAC OS

Programming Language :- Python

Libraries :- socket, tkinter, threading

Software used :- Visual Studio Code

IMPLEMENTATION

server_tcp_backend.py

```
# ================================
# TCP SERVER BACKEND (for GUI)
# File: server_tcp_backend.py
# ================================

import socket
import threading


class TCPServerBackend:
    def __init__(self, gui_callback):
        """
        gui_callback: function to update the GUI with new messages
        """
        self.gui_callback = gui_callback
        self.server_socket = None
        self.client_socket = None
        self.running = False

    def start_server(self, host="localhost", port=5050):
        """Start the TCP server"""
        self.running = True

        try:
            self.server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
            self.server_socket.bind((host, port))
            self.server_socket.listen(1)
            self.gui_callback(f"[TCP] Server started on {host}:{port}\nWaiting for client...\n")

            threading.Thread(target=self.accept_connection, daemon=True).start()

        except Exception as e:
            self.gui_callback(f"[ERROR] Could not start TCP server: {e}\n")

    def accept_connection(self):
        """Wait for client to connect"""
        try:
            self.client_socket, addr = self.server_socket.accept()
            self.gui_callback(f"[TCP] Client connected: {addr}\n")

            threading.Thread(target=self.receive_messages, daemon=True).start()

        except Exception as e:
            self.gui_callback(f"[ERROR] Accept failed: {e}\n")
```

```
    def receive_messages(self):
        """Receive messages from client"""
        while self.running:
            try:
                msg = self.client_socket.recv(1024).decode()
                if not msg:
                    break
                self.gui_callback(f"Client: {msg}\n")
            except:
                break

        self.gui_callback("[TCP] Client disconnected.\n")

    def send_message(self, message):
        """Send message to connected client"""
        if self.client_socket:
            try:
                self.client_socket.send(message.encode())
            except:
                self.gui_callback("[ERROR] Failed to send message.\n")

    def stop_server(self):
        """Stop the server and close sockets"""
        self.running = False

        try:
            if self.client_socket:
                self.client_socket.close()
            if self.server_socket:
                self.server_socket.close()
        except:
            pass

        self.gui_callback("[TCP] Server stopped.\n")
```

server_udp_backend.py

```
# ====================================
# UDP SERVER BACKEND (for GUI)
# File: server_udp_backend.py
# ====================================

import socket
import threading


class UDPServerBackend:
    def __init__(self, gui_callback):
        """
        gui_callback: function to update GUI with new messages
        """
        self.gui_callback = gui_callback
```

```
        self.server_socket = None
        self.client_addr = None
        self.running = False

    def start_server(self, host="localhost", port=6060):
        """Start UDP server"""
        self.running = True

        try:
            self.server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
            self.server_socket.bind((host, port))

            self.gui_callback(f"[UDP] Server started on {host}:{port}\nWaiting for client...\n")

            threading.Thread(target=self.receive_messages, daemon=True).start()

        except Exception as e:
            self.gui_callback(f"[ERROR] Could not start UDP server: {e}\n")

    def receive_messages(self):
        """Receive messages + store client address"""
        while self.running:
            try:
                data, addr = self.server_socket.recvfrom(1024)

                if self.client_addr is None:
                    self.client_addr = addr
                    self.gui_callback(f"[UDP] Client connected: {addr}\n")

                msg = data.decode()
                self.gui_callback(f"Client: {msg}\n")

            except Exception:
                break

        self.gui_callback("[UDP] Server stopped receiving.\n")

    def send_message(self, message):
        """Send message to client (UDP needs client_addr first)"""
        if self.client_addr:
            try:
                self.server_socket.sendto(message.encode(), self.client_addr)
            except:
                self.gui_callback("[ERROR] Failed to send message.\n")
        else:
            self.gui_callback("[!] No client yet. Client must send a message first.\n")

    def stop_server(self):
        self.running = False
        try:
            if self.server_socket:
                self.server_socket.close()
        except:
```

```
        pass

        self.gui_callback("[UDP] Server stopped.\n")
```

server_gui.py

```
# ============================
# SERVER GUI (TCP + UDP)
# File: server_gui.py
# ============================

import tkinter as tk
from tkinter import ttk
import threading

from server_tcp_backend import TCPServerBackend
from server_udp_backend import UDPServerBackend


class ServerGUI:
    def __init__(self):
        self.window = tk.Tk()
        self.window.title("Server Chat (TCP/UDP)")
        self.window.geometry("500x600")

        # Message display
        self.chat_box = tk.Text(self.window, height=25, width=60, state="disabled")
        self.chat_box.pack(pady=10)

        # Protocol selection
        tk.Label(self.window, text="Select Protocol:").pack()
        self.protocol = ttk.Combobox(self.window, values=["TCP", "UDP"], state="readonly")
        self.protocol.current(0)
        self.protocol.pack(pady=5)

        # Buttons
        self.start_btn = tk.Button(self.window, text="Start Server", command=self.start_server)
        self.start_btn.pack(pady=5)

        self.stop_btn = tk.Button(self.window, text="Stop Server", command=self.stop_server)
        self.stop_btn.pack(pady=5)

        # Message area
        tk.Label(self.window, text="Type your message below:").pack()

        self.input_frame = tk.Frame(self.window, bd=2, relief="groove")
        self.input_frame.pack(pady=5, fill="x")

        self.msg_entry = tk.Entry(self.input_frame, width=40)
        self.msg_entry.insert(0, "Enter your message...")
        self.msg_entry.pack(side="left", padx=5, pady=5)
```

```
        self.send_btn = tk.Button(self.input_frame, text="Send", bg="lightblue",
command=self.send_message)
        self.send_btn.pack(side="right", padx=5)

        # backend refs
        self.tcp_server = None
        self.udp_server = None

    def update_chat(self, message):
        self.chat_box.config(state="normal")
        self.chat_box.insert("end", message)
        self.chat_box.see("end")
        self.chat_box.config(state="disabled")

    def start_server(self):
        proto = self.protocol.get()

        if proto == "TCP":
            self.tcp_server = TCPServerBackend(self.update_chat)
            threading.Thread(target=self.tcp_server.start_server, daemon=True).start()

        else:
            self.udp_server = UDPServerBackend(self.update_chat)
            threading.Thread(target=self.udp_server.start_server, daemon=True).start()

        self.update_chat(f"[INFO] {proto} Server Started.\n")

    def send_message(self):
        message = self.msg_entry.get()
        if not message.strip():
            return

        proto = self.protocol.get()

        self.update_chat(f"Server: {message}\n")

        if proto == "TCP" and self.tcp_server:
            self.tcp_server.send_message(message)

        elif proto == "UDP" and self.udp_server:
            self.udp_server.send_message(message)

        self.msg_entry.delete(0, "end")

    def stop_server(self):
        proto = self.protocol.get()

        if proto == "TCP" and self.tcp_server:
            self.tcp_server.stop_server()

        elif proto == "UDP" and self.udp_server:
            self.udp_server.stop_server()
```

```
    def run(self):
        self.window.mainloop()


if __name__ == "__main__":
    ServerGUI().run()
```

client_gui.py

```
# ==============================
# CLIENT GUI (TCP + UDP)
# File: client_gui.py
# ==============================

import tkinter as tk
from tkinter import ttk
import socket
import threading


class ClientGUI:
    def __init__(self):
        self.window = tk.Tk()
        self.window.title("Client Chat (TCP/UDP)")
        self.window.geometry("500x600")

        # chatbox
        self.chat_box = tk.Text(self.window, height=25, width=60, state="disabled")
        self.chat_box.pack(pady=10)

        # protocol
        tk.Label(self.window, text="Select Protocol:").pack()
        self.protocol = ttk.Combobox(self.window, values=["TCP", "UDP"], state="readonly")
        self.protocol.current(0)
        self.protocol.pack(pady=5)

        # connect button
        self.connect_btn = tk.Button(self.window, text="Connect", command=self.connect)
        self.connect_btn.pack(pady=5)

        # disconnect button
        self.disconnect_btn = tk.Button(self.window, text="Disconnect", command=self.disconnect)
        self.disconnect_btn.pack(pady=5)

        # Message input
        tk.Label(self.window, text="Type your message below:").pack()

        self.input_frame = tk.Frame(self.window, bd=2, relief="groove")
        self.input_frame.pack(pady=5, fill="x")

        self.msg_entry = tk.Entry(self.input_frame, width=40)
        self.msg_entry.insert(0, "Enter your message...")
        self.msg_entry.pack(side="left", padx=5, pady=5)
```

```
        self.send_btn = tk.Button(self.input_frame, text="Send", bg="lightblue",
command=self.send_message)
        self.send_btn.pack(side="right", padx=5)

        # connection vars
        self.tcp_socket = None
        self.udp_socket = None
        self.running = False

    def update_chat(self, msg):
        self.chat_box.config(state="normal")
        self.chat_box.insert("end", msg)
        self.chat_box.see("end")
        self.chat_box.config(state="disabled")

    def connect(self):
        proto = self.protocol.get()

        if proto == "TCP":
            self.tcp_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
            self.tcp_socket.connect(("localhost", 5050))
            self.update_chat("[TCP] Connected to server.\n")

            self.running = True
            threading.Thread(target=self.receive_tcp, daemon=True).start()

        else:
            self.udp_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
            self.update_chat("[UDP] Ready. Send first message to create connection.\n")

            self.running = True
            threading.Thread(target=self.receive_udp, daemon=True).start()

    def send_message(self):
        msg = self.msg_entry.get()
        if not msg.strip():
            return

        proto = self.protocol.get()

        self.update_chat(f"You: {msg}\n")

        if proto == "TCP" and self.tcp_socket:
            self.tcp_socket.send(msg.encode())

        elif proto == "UDP" and self.udp_socket:
            self.udp_socket.sendto(msg.encode(), ("localhost", 6060))

        self.msg_entry.delete(0, "end")

    def receive_tcp(self):
        while self.running:
```

```
            try:
                data = self.tcp_socket.recv(1024).decode()
                if not data:
                    break
                self.update_chat(f"Server: {data}\n")
            except:
                break

    def receive_udp(self):
        while self.running:
            try:
                data, _ = self.udp_socket.recvfrom(1024)
                self.update_chat(f"Server: {data.decode()}\n")
            except:
                break

    def disconnect(self):
        self.running = False

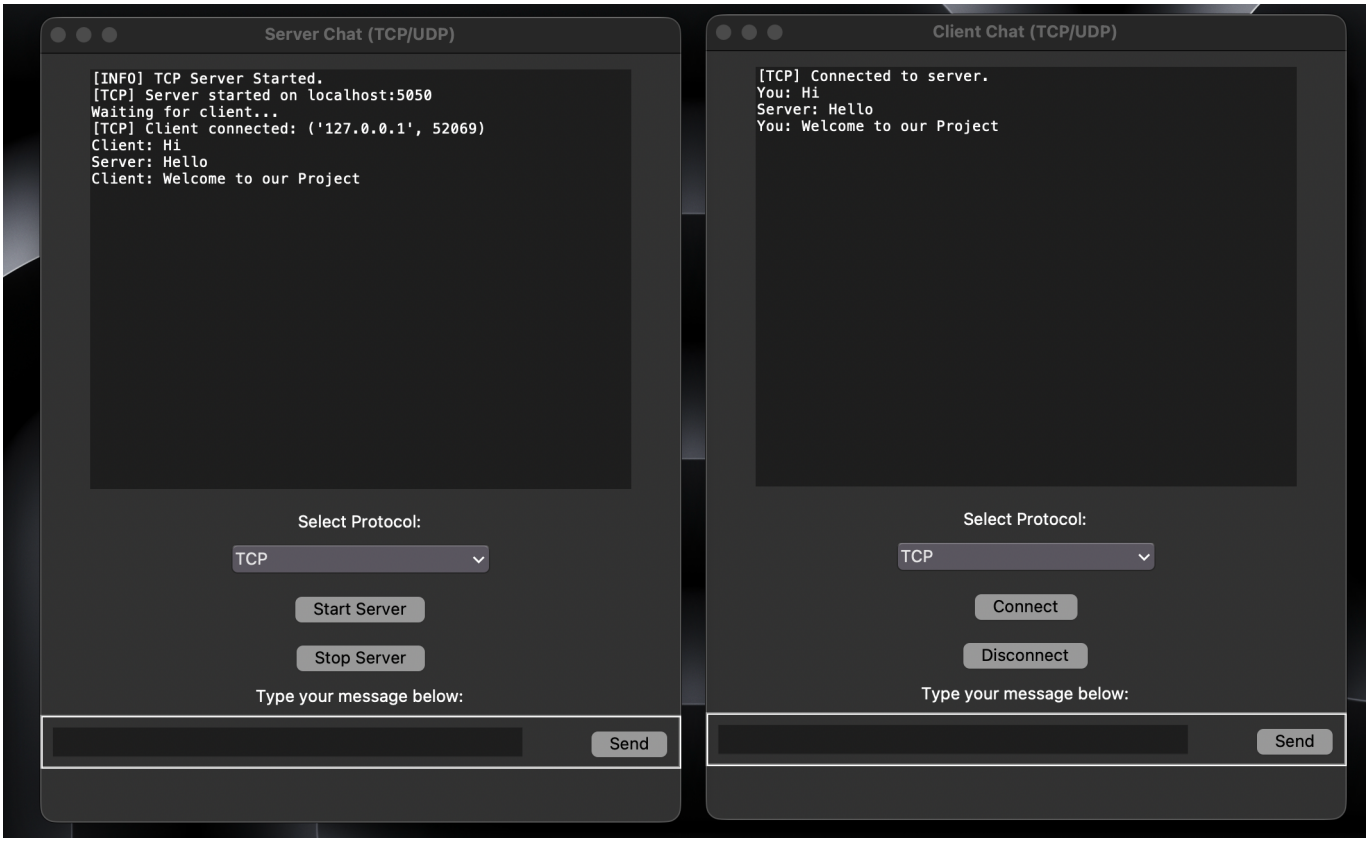
        if self.tcp_socket:
            self.tcp_socket.close()
        if self.udp_socket:
            self.udp_socket.close()

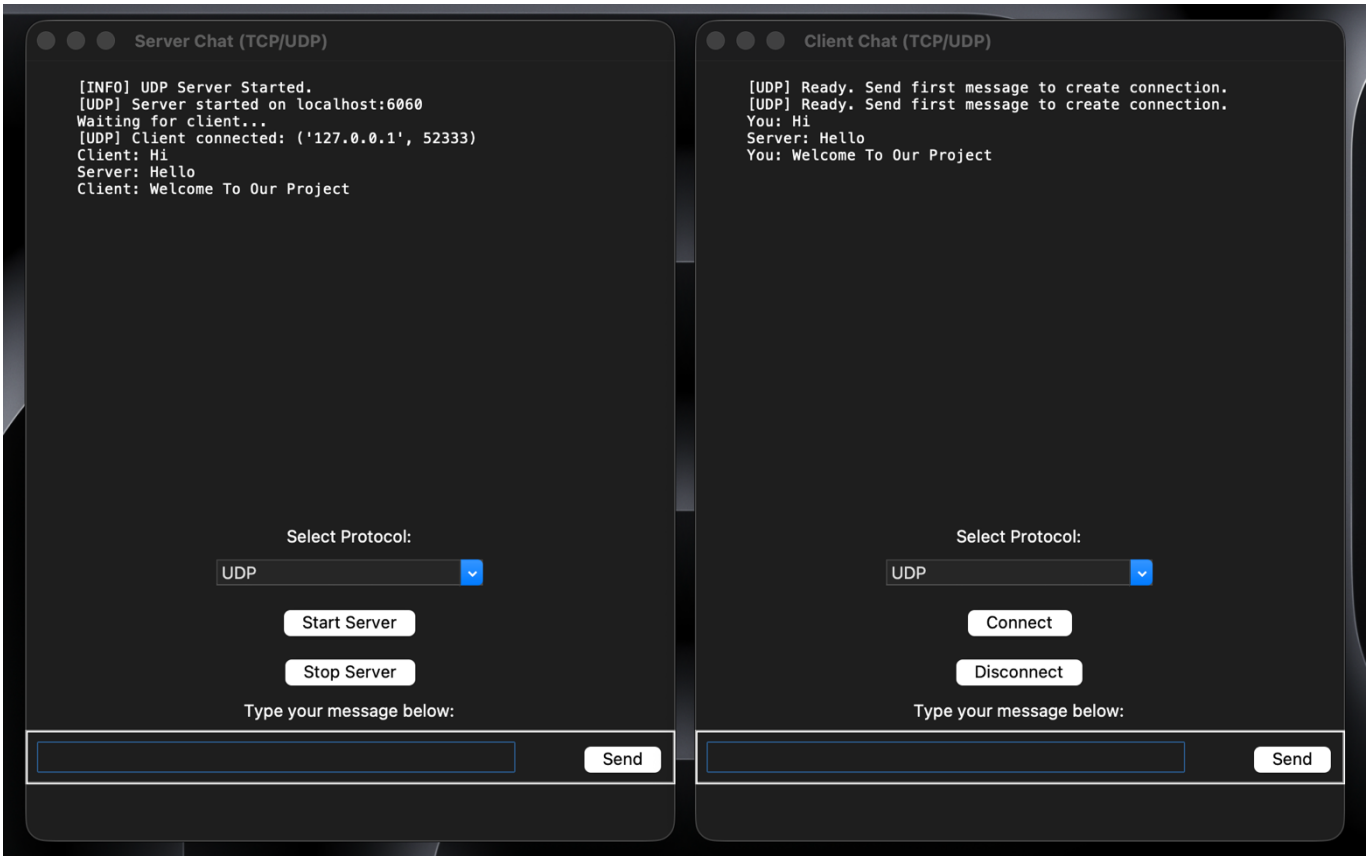
        self.update_chat("[INFO] Disconnected.\n")

    def run(self):
        self.window.mainloop()


if __name__ == "__main__":
    ClientGUI().run()
```

RESULT


Server Chat (TCP/UDP)
[INFO] TCP Server Started.
[TCP] Server started on localhost:5050
Waiting for client...
[TCP] Client connected: ('127.0.0.1', 52069)
Client: Hi
Server: Hello
Client: Welcome to our Project
Select Protocol:
TCP
Start Server
Stop Server
Type your message below:
Send
Client Chat (TCP/UDP)
[TCP] Connected to server.
You: Hi
Server: Hello
You: Welcome to our Project
Select Protocol:
TCP
Connect
Disconnect
Type your message below:
Send


Server Chat (TCP/UDP)
[INFO] UDP Server Started.
[UDP] Server started on localhost:6060
Waiting for client...
[UDP] Client connected: ('127.0.0.1', 52333)
Client: Hi
Server: Hello
Client: Welcome To Our Project
Select Protocol:
UDP
Start Server
Stop Server
Type your message below:
Send
Client Chat (TCP/UDP)
[UDP] Ready. Send first message to create connection.
[UDP] Ready. Send first message to create connection.
You: Hi
Server: Hello
You: Welcome To Our Project
Select Protocol:
UDP
Connect
Disconnect
Type your message below:
Send

CONCLUSION

The TCP/UDP Chat Application with GUI successfully demonstrates how computer networks enable communication between two systems using both connection-oriented and connectionless protocols. By integrating socket programming with a graphical user interface, the project overcomes the limitations of traditional terminal-based chat programs and provides a more practical, interactive, and user-friendly experience.

The system allows users to clearly observe the behaviour of **TCP (reliable)** and **UDP (fast but connectionless)** communication, making it easier to understand the differences between the two protocols. The GUI-based design improves clarity by separating message display and input areas, offering an experience similar to real-world messaging applications.

Overall, this project enhances the learning experience by combining theoretical networking concepts with practical implementation. It also showcases how Python can be used to build real-time communication tools with minimal resources, proving effective for both educational purposes and small-scale applications.